

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки
кафедра програмної інженерії

Практичне завдання №1
з дисципліни «Основи програмної інженерії»

Робота над проектом «Replit»

Над проектом працювала:

Теребун Юлія Андріївна

Посилання на гітхаб:

<https://github.com/yuliaterebun-alt/SRS-Replit/blob/main/README.md>

Харків 2026

Індивідуальне завдання

Крок 1. ДОСЛІДЖЕННЯ (20 хвилин)

- Відкрийте посилання на сервіс-прототип, отримане від викладача.
- Зареєструйтеся (якщо потрібно). Дослідіть всі доступні функції.
- Занотуйте: хто є типовими користувачами? Яку проблему вирішує сервіс?
- Заповніть таблицю спостережень (Таблиця 1 нижче)

Дослідження

Replit — це хмарна платформа для спільної розробки, яка дозволяє писати, запускати та розгортати код безпосередньо у браузері без будь-якого попереднього налаштування на комп'ютері.

Типові користувачі:

- Студенти та початківці: ті, хто тільки вчиться програмувати й не хоче витрачати години на встановлення компіляторів, бібліотек чи налаштування локального середовища.
- Викладачі та ментори: вчителі шкіл, викладачі університетів та ментори курсів, які використовують платформу для перевірки домашніх завдань, проведення практичних занять та створення інтерактивних уроків.
- Розробники (для прототипування): досвідчені програмісти, яким потрібно швидко протестувати ідею, написати невеликий скрипт або поділитися шматочком коду (snippet) з колегами.
- Команди розробників: невеликі стартапи або хакатон-команди, які використовують функції спільного редагування коду в реальному часі.

Яку проблему вирішує сервіс?

"Пекло налаштувань": замість того, щоб годинами встановлювати Python, Node.js, бази даних чи змінні оточення, користувач отримує готове робоче середовище за один клік.

Слабке апаратне забезпечення: Оскільки весь код виконується на потужних хмарних серверах Replit, програмувати складні проєкти можна навіть зі старого ноутбука, планшета чи смартфона.

Складність хостингу та деплою: Replit дозволяє не просто написати код, а й миттєво опублікувати вебсайт чи бот у мережі Інтернет (хостинг в один клік).

Спільна робота (Колаборація): Спільне редагування коду в стилі Google Docs вирішує проблему синхронізації проєктів між учасниками команди.

Таблиця спостережень:

№	Функція/Екран/Дія	Що робить (опис)	Тип (Ф/НФ/О)
1	Створення проєкту	Дозволяє вибрати мову програмування (або шаблон) та миттєво створити ізольоване середовище для написання коду.	Ф
2	Редактор коду	Забезпечує підсвічування синтаксису, автодоповнення, перевірку помилок «на льоту» та навігацію за файловою структурою.	Ф
3	<u>Вбудований ІІІ</u>	<u>Автоматично дописує код, генерує фрагменти за текстовим описом та допомагає знаходити й виправляти баги.</u>	<u>Ф</u>
4	Спільний доступ	Надає можливість працювати над одним кодом кільком користувачам одночасно, бачити курсори колег та спілкуватися в чаті.	Ф
5	Інтеграція з Git / GitHub	Дозволяє клонувати репозиторії, фіксувати зміни (commit) та	Ф

		відправляти їх (push) на GitHub безпосередньо через інтерфейс.	
6	Швидкість розгортання додатка	Вебсайти, скрипти та боти запускаються в хмарі за лічені секунди без ручного налаштування серверів (висока продуктивність).	НФ
7	Адаптивність інтерфейсу та теми	Робочий простір легко масштабується, підтримує темний/ світлий режими та коректно відображається навіть на мобільних пристроях.	НФ
8	Обмеження ресурсів	Безкоштовний тариф надає лімітований обсяг оперативної пам'яті (RAM). процесора (CPU) та робить усі створені проекти публічними.	О
9	Система безпеки	забезпечує Корпоративний рівень безпеки: перевірка коду перед деплоєм, ізолювані пісочниці для AI-агентів і опції єдиного входу (SSO) для бізнес-акаунтів	НФ

Крок 2. АНАЛІЗ (15 хвилин)

- Класифікуйте функції: Ф (функціональна), НФ (нефункціональна), О (обмеження).

- Визначте акторів (хто використовує систему? незареєстрований / зареєстрований

/ адмін).

- Побудуйте список Use Cases (мінімум 5).

- Визначте пріоритети: Must Have / Should Have / Could Have (MoSCoW)

Класифікація функцій

Функціональна

1. Replit Agent (створення по промпту)
2. Інтерактивне полотно / візуальний редактор
3. Робочі процеси (Workflows)
4. Хмарна IDE (редактор + термінал)
5. Спільна робота в реальному часі
6. Розгортання (Deploy) в один клік
7. Готові інтеграції (OpenAI, Stripe та ін.)
8. Вбудована база даних (ReplDB)
9. Аутентифікація користувачів (replAuth)

Нефункціональна

1. Доступність з браузера і мобільного додатку
2. Підтримка 50 + мов програмування
3. Автоматичне масштабування при деплоє
4. Ізольовані пісочниці для AI-агентів
5. SSO/SAML
6. Soc2 відповідність
7. Пріоритетна підтримка (Pro-тариф)
8. Автоматичне збереження коду та бази даних

Обмеження

1. Ліміт: 3 публічних проекту на безкоштовному тарифі
2. Ліміт: до 5 учасників на Core-тарифі
3. Ліміт: до 15 учасників на Pro-тарифі
4. Обмеження по режимам (Economy / Power / Turbo)
5. Тільки публічні проекти на безкоштовному тарифі

Актори системи (веб сайту)

1. Незареєстрований користувач
2. Зареєстрований користувач
3. Адміністратор робочого простору

Use Cases

1. Реєстрація на сервісі (створення користувачем акаунту)
2. Вхід у систему (Користувач може зайти на вже існуючий акаунт)
3. Створення проекту вручну або через AI-агента
4. Додавання співавтора в проект при наявності
5. Публікація проекту
6. Використання вбудованої бази даних

Визначення пріоритетів

Must Have (обов'язково) / Should Have (бажано) / Could Have

(необов'язково)

Must Have (обов'язково)

1. Вхід в аккаунт (аутентифікація)
2. Хмарна IDE (редактор коду, термінал, консоль)
3. Створення проекту (вручну або через AI-агента)
4. Збереження проекту та контроль версій (знімки)
5. Розгортання (деплой) з отриманням публічного посилання

Should Have (бажано)

1. Спільна робота в реальному часі
2. Вбудована база даних (ReplDB)

3. Клонування та ремікс проектів
4. Інструменти налагодження (Debugging)
5. Система аутентифікації користувачів (replAuth)

Could Have (необов'язково)

1. Режим "Спостерігач" (Observer Mode)
2. Готові інтеграції (OpenAI, Stripe та ін.)
3. Інтерактивне полотно (візуальний редактор)
4. Робочі процеси (Workflows)
5. Аналітика опублікованих проектів

Крок 3. НАПИСАННЯ SRS (30 хвилин)

- Заповніть шаблон SRS (Додаток Б) на основі аналізу.
- Мінімальний обсяг: 1–2 сторінки. Кожна вимога — окремий пронумерований пункт.
- Перевірте вимоги за критерієм SMART-R (розд. 1.3).

SRS (Software Requirements Specification)

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Специфікація вимог до програмного забезпечення

Поле	Replit
Назва проекту	Replit - платформа для розробки та розгортання додатків
Версія документа	1.0
Дата	27.06.2026
Команда	1 студентка - Теробун Юлія Андріївна
Замовник	Бізнес-підрозділи та стартапи

ВСТУП (INTRODUCTION)

Replit - це ai-орієнтована платформа для створення програмного забезпечення, яка об'єднує в собі браузерну IDE, хостинг, бази даних, аутентифікацію і розгортання в єдиному середовищі. На відміну від інших ai-інструментів для програмування, таких як Github Copilot або Cursor, які орієнтовані на професійних розробників, Replit позиціонує себе як повнофункціональне рішення для "знаючих фахівців" (knowledge workers), що не вимагає глибоких технічних навичок.

Проблема та рішення(функції): традиційна розробка програмного забезпечення вимагає значних ресурсів: найму інженерів, налаштування інфраструктури (сервери, бази даних, аутентифікація), управління DevOps-процесами і тривалих циклів розробки. Для "не-технічних" фахівців (менеджерів, маркетологів, операційних команд) створення навіть простого додатка або внутрішнього інструменту стає непереборним бар'єром. Платформа надає "все в одному" - від ідеї до запущеного додатка, міняючи традиційні етапи найму розробників, налаштування інфраструктури та DevOps.

Замовник: Бізнес-підрозділи та "не-розробники"

Цільова аудиторія: Клієнти цих закладів (будь якого віку)

ЗАГАЛЬНИЙ ОПИС (OVERALL DESCRIPTION)

Актори

1. Незареєстрований користувач (переглядає публічні проекти та документацію)
2. Зареєстрований користувач (створює та редагує проекти)
3. Учасник робочого простору
4. Адміністратор (має повний доступ до налаштувань організації, управління учасниками)
5. Менеджер (керує учасниками та групами)
6. Редактор (редагує код)

Залежності

1. Браузер (Google Chrome, Firefox, Safari, Edge)
2. Стабільне Інтернет-з'єднання
3. Обліковий запис Replit (для створення та редагування проектів)
4. Браузерний JavaScript (включений)
5. Підтримка WebSocket (для спільної роботи в реальному часі)
6. Вбудована база даних ReplDB (для зберігання даних програми)

7. Система аутентифікації replAuth (для управління користувачами)

ФУНКЦІОНАЛЬНІ ВИМОГИ (FUNCTIONAL REQUIREMENTS)

FR-01 [must]: Користувач повинен мати можливість створити проект через AI-агента з опису природною мовою. Система повинна генерувати працюючий код, налаштовувати оточення і встановлювати залежності автоматично.

FR-02 [must]: Користувач повинен мати доступ до хмарної IDE за допомогою редактора коду, терміналу та консолі.

FR-03 [must]: Система повинна надавати можливість розгортання (деплой) додатка в один клік з отриманням публічного посилання.

FR-04 [must]: Користувач повинен мати можливість зберігати проект і керувати версіями через автоматичні знімки (Snapshots) для відкоту до попередніх станів.

FR-05 [must]: Система повинна підтримувати вбудовану базу даних для зберігання даних програми.

FR-06 [should]: Користувач повинен мати можливість реєстрації та автентифікації через email, Google або GitHub.

FR-07 [should]: Система повинна підтримувати режими роботи AI-агента.

FR-08 [should]: Користувач повинен мати можливість підключати готові інтеграції (OpenAI, Stripe, Google Workspace та ін.) без необхідності введення API-ключів.

FR-09 [should]: Система повинна підтримувати режим "спостерігач".

FR-10 [could]: Система повинна забезпечити інтерактивне Полотно для візуального редагування інтерфейсу.

FR-11 [could]: Система повинна надавати аналітику для опублікованих проектів.

НЕФУНКЦІОНАЛЬНІ ВИМОГИ (NON-FUNCTIONAL REQUIREMENTS)

Продуктивність (Performance)

NFR-P-01: Система повинна забезпечувати миттєвий запуск робочого простору і готовність середовища розробки до використання протягом декількох секунд після створення або відкриття проекту .

NFR-P-02: Система повинна забезпечувати мінімальну затримку при роботі AI-агента і автодоповнень коду, підтримуючи режим Turbo для прискорення до 2 разів.

NFR-P-03: ресурсні квоти на проект повинні визначатися тарифним планом.

NFR-P-04: Система повинна забезпечувати швидкі операції копіювання (Remix) і відновлення (Snapshot) файлових систем і баз даних.

NFR-P-05: Платформа повинна підтримувати мінімальну затримку при виконанні операцій з базою даних в dev-оточенні, забезпечуючи комфортну розробку.

Безпека (Security)

NFR-S-01: Система повинна реалізовувати автентифікацію та авторизацію між усіма внутрішніми службами за допомогою короткострокових токенів.

NFR-S-02: Платформа повинна забезпечувати ізоляцію середовищ розробки.

NFR-S-03: Система повинна забезпечувати повне розділення середовищ розробки та виробництва.

NFR-S-04: Платформа повинна забезпечувати безперервне резервне копіювання файлових систем.

NFR-S-05: Система повинна виконувати сканування коду на вразливості перед публікацією.

Зручність використання (Usability)

NFR-U-01: Система повинна забезпечити інтуїтивно зрозумілий інтерфейс.

NFR-U-02: Платформа повинна підтримувати роботу в будь-якому сучасному браузері без необхідності установки додаткового ПЗ.

NFR-U-03: Система повинна забезпечувати узгоджене оточення для всіх учасників команди.

NFR-U-04: Платформа повинна підтримувати співпрацю в режимі реального часу.

NFR-U-05: Система повинна надавати візуальний інтерфейс управління версіями з можливістю відкату до будь-якої попередньої версії.

Надійність (Reliability)

NFR-R-01: Платформа повинна забезпечувати високу доступність середовища розробки та публікації.

NFR-R-02: Система повинна забезпечити механізм автоматичного відновлення.

NFR-R-03: Платформа повинна підтримувати механізм перевірки коду та бази даних з можливістю швидкого відновлення та відкату змін.

NFR-R-04: Система повинна забезпечувати надійність зберігання даних.

Масштабованість (Scalability)

C-01: Платформа повинна підтримувати як комп'ютерний, так і мобільний формат.

C-02: Система повинна забезпечувати горизонтальне масштабування розгорнутих додатків.

C-03: Платформа повинна підтримувати масштабування сховища.

C-04: Система повинна забезпечувати масштабування командної роботи.

ГЛОСАРІЙ (GLOSSARY)

Хмарна IDE - браузерна середовище розробки з редактором коду, терміналом, консоллю і відладчиком, доступна без установки ПО

Снапшот - Автоматичний "знімок" стану проекту (коду і бази даних). Дозволяє відкотитися до будь-якої попередньої версії.

Ремікс - створення власної копії існуючого публічного проекту.

QR-код - двовимірний штрихкод, що містить закодовану інформацію (посилання, текст, контакти).

Деплой (deploy) - процес розміщення і запуску готового додатка на сервері, що робить його доступним для користувачів через Інтернет за публічним посиланням.

Аутентифікація - процес перевірки автентичності користувача при вході в систему.

База даних (БД) -структуроване сховище інформації, яке дозволяє зберігати, змінювати і отримувати дані при роботі платформи.

МАТРИЦЯ ВІДСТЕЖУВАНOSTІ (TRACEABILITY MATRIX)

FR-01 → 2.1 → Тест: Користувач вводить текстовий опис програми і система генерує робочий код
FR-03 → Створення меню → Тест: додавання меню

FR-03 → 2.2 → Тест: Користувач натискає кнопку "Deploy" і додаток стає доступним за публічним посиланням

FR-06 → 2.3 → Тест: реєстрація користувача

ОПИТУВАЛЬНИК ДЛЯ ІНТЕРВ'Ю ІЗ ЗАМОВНИКОМ

1.1. Опишіть одним реченням: для чого призначений цей продукт / сервіс?

Відповідь: Платформа для створення, розгортання та хостингу додатків та веб-сайтів.

1.2. Хто є основними користувачами? (Вік, роль, технічний рівень)

Відповідь: основні користувачі - це засновники стартапів, продакт-менеджери, тобто не професійні розробники.

1.3. Яку проблему вирішує продукт? Чому існуючі рішення не задовольняють?

Відповідь: вирішує проблему недоступності розробки ПЗ для бізнес-користувачів.

1.4. Яка кінцева бізнес-мета? (Прибуток, зменшення витрат, соціальний ефект?)

Відповідь: Кінцева бізнес-мета-економія витрат клієнтів на найм розробників, а також соціальний ефект від демократизації створення ПЗ.

1.5. Чи є конкуренти / аналоги? Чим ваш продукт відрізняється?

Відповідь: Конкуренти - Github Copilot, Cursor, Figma. Різниця: це єдина платформа "все в одному" без необхідності Налаштування, орієнтована не на інженерів, а на бізнес-користувачів.

ФУНКЦІОНАЛЬНІ ВИМОГИ

2.1. Перерахуйте основні дії, які користувач повинен мати можливість виконати в системі (мінімум 5).

Відповідь: створення проекту через ai-агента з опису, редагування коду в хмарній IDE, розгортання (деплой) в один клік, запрошення співавторів для спільної роботи, управління версіями через знімки (Snapshots).

2.2. Які функції є обов'язковими (без них продукт не має сенсу)? Які — бажаними?

Відповідь: Обов'язкові: хмарна IDE, створення проекту, деплой, збереження/контроль версій. Бажані: AI-агент, спільна робота, вбудована БД і аутентифікація, інтеграції.

2.3. Чи є різні ролі користувачів? (неzareєстрований / zareєстрований / адміністратор). Чи відрізняються їх права?

Відповідь: незареєстрований-тільки перегляд публічних проектів. Зареєстрований-створення та редагування проектів, деплой, запрошення співавторів. Адміністратор-управління учасниками, доступом та безпекою.

2.4. Опишіть типовий сценарій використання: що робить користувач від першого запуску до досягнення мети?

Відповідь: реєстрація → створення проекту (через AI-агента або вручну) → редагування коду/дизайну → налаштування БД та інтеграцій → запрошення співавторів (опціонально) → розгортання → отримання публічного посилання → моніторинг.

2.5. Які дані система зберігає? Що показує користувачу? Які повідомлення / сповіщення відправляє?

Відповідь: Зберігає: код проектів, файли, БД, знімки версій, облікові дані, Налаштування оточення, метадані користувачів. Показує: редактор, термінал, структуру проекту, статус деплоя, аналітику. Повідомляє: про помилки збірки, Запрошення в проекти, зміни в спільній роботі, закінчення кредитів.

НЕФУНКЦІОНАЛЬНІ ВИМОГИ

3.1. Скільки одночасних користувачів очікується? Яке навантаження?

Відповідь: очікуються сотні і тисячі користувачів. База даних з пулом з'єднань витримує близько 400 одночасних користувачів.

3.2. Яке допустиме максимальне time-to-response для типових операцій? (1с? 3с?)

Відповідь: для AI-функцій час відгуку значно скорочено за рахунок поліпшень інфраструктури. Час відповіді для критичних інцидентів: P1 (повний збій) — <15 хвилин, P2 (деградація) — <1 години.

3.3. Які вимоги до безпеки? (Аутентифікація, шифрування даних, роль GDPR/ССРА?)

Відповідь: підтримується автентифікація, шифрування, відповідність GDPR та ССРА. Дані обробляються відповідно до DPA. Доступні інструменти для реалізації запитів суб'єктів даних (експорт/видалення).

3.4. На яких пристроях і браузерях система має працювати? (Mobile-first? Desktop?)

Відповідь: працює в сучасних браузерах на десктопі і мобільних пристроях (iOS, Android) з підтримкою масштабування під мобільні екрани і QR-кодом для тестування.

3.5. Яка доступність системи (uptime)? Чи є критичні години роботи?

Відповідь: доступність забезпечується через професійний хостинг. Вбудована аналітика відстежує uptime і час відповіді. Логи зберігаються 7 днів.

ОБМЕЖЕННЯ ТА РИЗИКИ

4.1. Які є технологічні обмеження? (Мова програмування, фреймворк, хмара / on-premise?)

Відповідь: технологічні обмеження: платформа працює в хмарі (Google Cloud), підтримує 50+ мов, розгортання тільки в хмарі (без on-premise).

4.2. Який бюджет та дедлайн проекту?

Відповідь: Бюджет залежить від тарифу: Starter — безкоштовно, Core — \$20/міс (до 5 учасників), Pro — \$100/міс (до 15 учасників). Дедлайн не фіксований-у міру розробки.

4.3. Чи є регуляторні / правові обмеження? (Ліцензії, вікові обмеження, геолокація?)

Відповідь: вікове обмеження-13 + років (за згодою батьків для молодших), ліцензійні угоди (Terms of Service), заборона на некомерційне використання для безкоштовного тарифу, обмеження по геолокації відсутні.

4.4. Які залежності від зовнішніх систем / API?

Відповідь: залежить від зовнішніх систем: вбудовані інтеграції з OpenAI, Stripe, Google Workspace та ін.; хостинг на Google Cloud Platform (GCP); DNS-фільтрація через Cloudflare; управління пакетами через Nix; сторонні сервіси для SSO/SAML в Enterprise.

УТОЧНЕННЯ ТА ПРІОРИТИЗАЦІЯ

5.1. Якщо б ви могли реалізувати тільки ОДНУ функцію, яка б це була і чому?

Відповідь: Replit Agent, так як він є основою платформи, що перетворює бізнес-користувачів в творців ПЗ.

5.2. Що є абсолютно неприйнятним у поведінці системи? (Критичні «по-go»)

Відповідь: несанкціоноване зміна або видалення даних в продакшн-оточенні (особливо БД) без підтвердження і неможливість відкату. Впроваджено автоматичне розділення dev / prod БД.

5.3. Як ви визначатимете успіх цього продукту через 6 місяців?

Відповідь: зростання числа платять користувачів, стабільна робота без збоїв.

БЛАНК САМОРЕФЛЕКСІЇ (ІНДИВІДУАЛЬНИЙ)

Заповнюється кожним студентом.

ПІБ студента / Група	Теребун Юлія Андріївна
Назва досліджуваного сервісу	Replit
Дата виконання	30.05.2026

Що виявилось найскладнішим при формулюванні вимог?

Найскладнішим виявилось поділ функціональних можливостей AI-агента (які постійно оновлюються) і базових функцій платформи.

Яка різниця між функціональною та нефункціональною вимогою на КОНКРЕТНОМУ прикладі з вашого SRS?

Функціональна вимога описує що система повинна робити, а нефункціональна то, як конкретно відбувається даний процес.

Чи є у вашому SRS суперечності між вимогами? (Наприклад: «Максимальна безпека» vs «Мінімальний час завантаження»). Як їх можна вирішити?

Наприклад, безперервне резервне копіювання може конфліктувати миттєвим запуском робочого простору, так як створення знімків стану споживає ресурси в тлі.

Можна виконувати резервне копіювання асинхронно у фоновому режимі, а також обмежувати частоту створення снапшотів (не рідше рази на добу), щоб знизити навантаження на систему.

Що б ви змінили у процесі збору вимог, якби мали більше часу?

Я б провела інтерв'ю з користувачами даної платформи для того, щоб докладніше вивчити їх досвід і критерії успішної роботи. А також провела б більш детальний аналіз конкурентів цього сервісу для більш ефективного його розвитку.

Яку одну вимогу з вашого SRS ви вважаєте найціннішою для замовника і чому?

FR-01 є найціннішим, оскільки саме ця функція перетворює платформу з інструменту розробника на продукт для бізнес-користувачів, усуваючи головний бар'єр (необхідність знати програмування)